Temporal Time-Travel AQL Queries

All queries return `path` objects (`p.vertices[] + p.edges[]` with all `_id` / `_from` / `_to` fields populated). Paste each into ArangoDB web UI → **Query** tab → Run → click the **Graph** tab to render the interactive subgraph.

Temporal data model

Collection	Role
`DesignEpoch`	A named period of design history (git tag, major refactor, initial commit). Fields: `git_tag` , `epoch_type` , `start_ts` , `end_ts` (Unix timestamps), `start_commit` , `end_commit` .
`RTL_Module`	A **snapshot** of a Verilog module at a specific commit. Multiple documents can share the same `label` — each is a distinct version. Fields: `label` , `file_hash` , `valid_from_ts` , `valid_to_ts` (`9999999999` = still active), `design_epoch` .
`BELONGS_TO_EPOCH`	Connects `RTL_Module → DesignEpoch` . Edge field `role` is `"introduced_in"` for the first version, `"modified_in"` for subsequent changes.
`GitCommit`	A git commit node. Fields: `metadata.author` , `metadata.message` , `valid_from_ts` .
`MODIFIED`	Connects `GitCommit → RTL_Module` . Edge field `valid_from_ts` is the commit timestamp.
`CROSS_REPO_EVOLVED_FROM`	Connects a golden entity in one repo to its architectural predecessor in another. Fields: `lineage` , `confidence` .

MOR1KX milestone timeline (for reference):

git tag	year	`_id`
initial_commit	2012	`DesignEpoch/77f096fb68c3eb90`
1.0	2013	`DesignEpoch/e8e051bf6043558a`
5.0	2017	`DesignEpoch/05174b226ebb6b9f`
5.0.r2	2017	`DesignEpoch/6cb4a54f1b52ca81`
5.1	2022	`DesignEpoch/70f4715e0bdddbca`
5.1.1	2022	`DesignEpoch/2203d364bbb065a5`
5.2	2024	`DesignEpoch/728c71b36ab98f8a`

Query 1 — Module time-travel: same module across 4 milestones

Demonstrates: `mor1kx_cpu_cappuccino` evolved over 9 years (2013 → 2022). Each result node is the same logical module at a different point in design history. The changing `file_hash` on hover proves the implementation changed between versions.

Returns: 4 path objects — one `RTL_Module` snapshot per milestone epoch.

```
// ────────────────────────────────────────────────────────────────────────
// QUERY 1 — Module time-travel
//   Same module (mor1kx_cpu_cappuccino) at 4 milestone epochs: v1.0, v5.0, v5.0.r2, v5.1
//   Each RTL_Module node is a distinct snapshot; the file_hash changes prove evolution.
//   Node hover attributes to watch: file_hash, valid_from_ts, valid_to_ts, design_epoch
//   valid_to_ts = 9999999999 means the snapshot is the current/last known version.
//
// Graph tab shows: 4 RTL_Module nodes fanning out from 4 DesignEpoch nodes
// ────────────────────────────────────────────────────────────────────────
FOR epoch IN DesignEpoch
  FILTER epoch.repo == 'openrisc/mor1kx.git'
    AND epoch.epoch_type == 'milestone_tag'
  SORT epoch.start_ts ASC

  LET snap = FIRST(
    FOR mod IN INBOUND epoch BELONGS_TO_EPOCH
      FILTER mod.label == 'mor1kx_cpu_cappuccino'
      SORT mod.valid_from_ts DESC
      LIMIT 1
      RETURN mod
  )
  FILTER snap != null

  FOR epoch2, e, p IN 1..1 OUTBOUND snap BELONGS_TO_EPOCH
    FILTER epoch2._id == epoch._id
    RETURN p
```

Query 2 — Design snapshot: the entire MOR1KX 5.0.r2 design (2017)

Demonstrates: "What did the MOR1KX design look like at tag 5.0.r2 in December 2017?"
Every module that was active during that epoch fans out from the single epoch node.

Returns: 76 path objects — all `RTL_Module` snapshots at the `5.0.r2` milestone.

```
// ────────────────────────────────────────────────────────────────────────
// QUERY 2 — Design snapshot at MOR1KX 5.0.r2 (December 2017)
//   Anchor epoch: DesignEpoch/6cb4a54f1b52ca81 (git_tag = "5.0.r2")
//   All 76 RTL modules active in that milestone fan out from the single epoch node.
//
// Graph tab shows: one DesignEpoch hub with 76 RTL_Module spokes
// To narrow down, add a FILTER on mod.label (e.g. LIKE 'mor1kx_cpu%')
// ────────────────────────────────────────────────────────────────────────
FOR v, e, p IN 1..1 INBOUND 'DesignEpoch/6cb4a54f1b52ca81' BELONGS_TO_EPOCH
  RETURN p
```

To filter to a specific sub-system, add after `BELONGS_TO_EPOCH` :

```
  FILTER LIKE(v.label, 'mor1kx_cpu%') OR LIKE(v.label, 'mor1kx_ctrl%')
```

Query 3 — Commit-level time-travel: a 2021 Wishbone Interface refactor

Demonstrates: A single git commit (`"Formal: Wishbone Interface"` , Aug 2021, by Harshitha172000) touched 13 modules simultaneously. The commit node is the temporal anchor; each spoke is a module snapshot whose life began at exactly that commit's timestamp.

Returns: 13 path objects — `GitCommit` hub → 13 `RTL_Module` spokes via `MODIFIED` edges.

```
// ────────────────────────────────────────────────────────────────────────────
// QUERY 3 — Commit-level change event: "Formal: Wishbone Interface" (Aug 2021)
//   Anchor commit: GitCommit/f7f7931b3134e0740237242ff874dfb6820f52b4
//   Author: Harshitha172000 | timestamp: 2021-08-11
//   13 modules were modified in this single commit.
//
// Graph tab shows: one GitCommit hub with 13 RTL_Module spokes
// Node hover: file_hash, valid_from_ts, valid_to_ts (when each snapshot was superseded)
// ────────────────────────────────────────────────────────────────────────────
FOR v, e, p IN 1..1 OUTBOUND 'GitCommit/f7f7931b3134e0740237242ff874dfb6820f52b4' MODIFIED
  RETURN p
```

To see the first-ever MOR1KX commit instead (Julius Baxter, Jan 2012, "first commit", 19 modules):

```
FOR v, e, p IN 1..1 OUTBOUND 'GitCommit/15d78de71e714b4582675df422e87938f5b7be50' MODIFIED
  RETURN p
```

Query 4 — Evolutionary lineage: OR1200 → MOR1KX architectural succession

Demonstrates: Two layers of time-travel combined:

1. The `CROSS_REPO_EVOLVED_FROM` edge proves MOR1KX is the architectural successor of OR1200 (the same "OpenRISC 1000" concept lives in both golden entity layers, with `confidence: 0.9`).
2. The `BELONGS_TO_EPOCH` paths show `mor1kx_cpu` at the project's birth (2012) and its first stable release (v1.0, 2013), with distinct `file_hash` values confirming it changed.

Returns: 3 path objects — 1 cross-repo evolutionary bridge + 2 module milestone snapshots.

```
// ────────────────────────────────────────────────────────────────────────────
// QUERY 4 — Evolutionary lineage: OR1200 → MOR1KX (conceptual + RTL milestones)
//
// Part A: the golden-entity bridge
//   MOR1KX:OpenRISC 1000 —[CROSS_REPO_EVOLVED_FROM]→ OR1200:OpenRISC 1000
//   Edge fields: lineage = "direct_architectural_successor", confidence = 0.9
//
// Part B: RTL snapshots of mor1kx_cpu at initial_commit (2012) and v1.0 (2013)
//   Each is a BELONGS_TO_EPOCH path showing the module at a specific epoch
//
// Graph tab shows: 2 golden entity nodes connected by EVOLVED_FROM,
//                  plus 2 RTL_Module snapshots connected to their epoch nodes
// ────────────────────────────────────────────────────────────────────────────
FOR p IN UNION(

  // Part A: cross-repo evolutionary bridge
  (
    FOR ev, edge, path IN 1..1 OUTBOUND
        'MOR1KX_Golden_Entities/MOR1KX_g_2ccb86b96f39'
        CROSS_REPO_EVOLVED_FROM
      RETURN path
  ),

  // Part B: mor1kx_cpu at initial_commit + milestone v1.0
  (
    FOR epoch IN DesignEpoch
      FILTER epoch.repo == 'openrisc/mor1kx.git'
        AND epoch.epoch_type IN ['initial_commit', 'milestone_tag']
      SORT epoch.start_ts ASC
      LIMIT 4
      LET snap = FIRST(
        FOR mod IN INBOUND epoch BELONGS_TO_EPOCH
          FILTER mod.label == 'mor1kx_cpu'
          SORT mod.valid_from_ts DESC
          LIMIT 1
          RETURN mod
      )
      FILTER snap != null
      FOR epoch2, e, path IN 1..1 OUTBOUND snap BELONGS_TO_EPOCH
        FILTER epoch2._id == epoch._id
        RETURN path
  )

)
RETURN p
```

GraphRAG temporal query — concept provenance across design generations

The GraphRAG collections (`_Chunks` , `_Entities` , `_Golden_Entities`) have `doc_version` and `valid_from_epoch` fields that are reserved for future re-ingestion tracking. The temporal signal that **is** available now is the inter-repo semantic bridge: concepts documented in the 2001 OR1200 specification can be traced forward through `CROSS_REPO_SIMILAR_TO` edges to their counterparts in IBEX (2018+), MAROCCHINO (2017+), and MOR1KX (2012+).

Query: "Which architectural concepts from the 2001 OR1200 spec were still present in successor designs two decades later?"

Path:

`OR1200_Chunk → [MentionedIn] → OR1200_Entity → [Consolidates] → OR1200_Golden_Entity → [CROSS_REPO_SIMILAR_TO] → Successor Golden Enti`

Anchor: chunk 0 of `openrisc1200_spec.pdf` (the table of contents, dated "Rev. 0.7, Sep 6, 2001"), which mentions 12 top-level architectural concepts.

Returns: 8 paths — each concept that survived from the 2001 spec into at least one later repo (IBEX, MAROCCHINO, or MOR1KX). The `similarity_score` on the `CROSS_REPO_SIMILAR_TO` edge shows semantic distance across generations.

```
// ──────────────────────────────────────────────────────────────────────────
// GRAPHRAG TEMPORAL — concept provenance: 2001 OR1200 spec → successor repos
//
// Hop 1: OR1200_Chunk  —[OR1200_MentionedIn]—▷  OR1200_Entity
//          The chunk is from openrisc1200_spec.pdf Rev 0.7, September 2001
//          It lists 12 top-level architectural concepts in the table of contents
//
// Hop 2: OR1200_Entity —[OR1200_Consolidates]—▷  OR1200_Golden_Entity
//          Consolidates de-duplicates and lifts entities to the golden layer
//
// Hop 3: OR1200_Golden_Entity —[CROSS_REPO_SIMILAR_TO]—▷  Successor_Golden_Entity
//          Bridges to the same concept as it appears in IBEX / MAROCCHINO / MOR1KX
//          Edge attribute similarity_score (0–1) = semantic similarity between generations
//
// Graph tab shows: spec chunk as the root, fanning out through entity layers
//                  to successor repo golden entities (colour-coded by repo)
// Temporal reading: concepts at left = documented 2001; at right = implemented 2012–2024
//
// To explore a different chunk, change the anchor _id.
// Chunk indices: 0 = ToC (12 concepts), 3 = Debug unit section,
//                9 = I/O port specifications
// ──────────────────────────────────────────────────────────────────────────
FOR v, e, p IN 3..3
  ANY 'OR1200_Chunks/OR1200_dee4cc87128f2a9f'  // ← chunk 0: ToC / intro, Rev 0.7 Sep 2001
  GRAPH 'IC_Temporal_Knowledge_Graph'
  OPTIONS { uniqueVertices: 'path' }

  FILTER REGEX_REPLACE(p.edges[0]._id, '/.*', '') == 'OR1200_MentionedIn'
  FILTER REGEX_REPLACE(p.edges[1]._id, '/.*', '') == 'OR1200_Consolidates'
  FILTER REGEX_REPLACE(p.edges[2]._id, '/.*', '') == 'CROSS_REPO_SIMILAR_TO'

  LIMIT 20
  RETURN p
```

What to look for in the visualizer:

Node	What it tells you
`OR1200_Chunk`	The 2001 spec as the temporal origin
`OR1200_Entity`	Concept as extracted from that document
`OR1200_Golden_Entity`	De-duplicated, canonical form of the concept
Successor golden entity	The same concept re-appearing in a later repo
`CROSS_REPO_SIMILAR_TO` edge	Hover for `similarity_score` — the semantic distance across time

The 8 surviving concepts are: **WISHBONE Interfaces**, **Debug unit** (→ IBEX), **Programmable Interrupt Controller** (→ IBEX, MAROCCHINO), **Data MMU** (→ MAROCCHINO, MOR1KX), and **OpenRISC 1200** (the whole architecture → MOR1KX).

Interval-based queries

These queries use explicit time ranges (`T1` / `T2` Unix timestamps) rather than epoch identifiers.
Edit the `LET T` / `LET T1` / `LET T2` lines to move through history.

Useful MOR1KX timestamp reference

Event	Date	Unix timestamp
First commit (project birth)	2012-01-26	`1327599687`
v1.0 release	2013-09-02	`1378100000`
v5.0 release	2017-04-27	`1493300000`
v5.0.r2 release	2017-12-14	`1513174055`
v5.1 release	2022-01-09	`1641710675`
v5.1.1 release	2022-05-22	`1653253288`
v5.2 release	2024-08-25	`1724578502`

Module count at each era: 24 (2012) → 37 (2013) → 48 (2017, stable through 2024).

Interval Query A — Point-in-time snapshot: "what did the design look like on a given date?"

Filters `RTL_Module` on `valid_from_ts <= T AND valid_to_ts >= T` to find every module that was active at exactly that moment. The `closest-epoch` dedup selects one epoch per module so each node appears once in the graph.

Demonstrates: "As of 2 September 2013 (just after v1.0) there were 37 active modules."
Change `T` to any other timestamp to travel to that point.

Returns: one path per active module — `RTL_Module → BELONGS_TO_EPOCH → DesignEpoch` .

```
// ────────────────────────────────────────────────────────────────────────────
// INTERVAL QUERY A — Point-in-time design snapshot
//   Change T to any Unix timestamp to "visit" the design at that moment.
//
//   Useful values for T (MOR1KX):
//     1327599687  project birth (2012-01-26)
//     1378100000  just after v1.0 release (2013-09-02)  → 37 modules
//     1493300000  v5.0 era (2017-04-27)                 → 48 modules
//     1641710675  v5.1 release (2022-01-09)             → 48 modules
//
// Graph tab shows: active RTL_Module nodes fanning out to their epoch hubs
// ────────────────────────────────────────────────────────────────────────────
LET T = 1378100000   // ← change this timestamp to travel through history

FOR mod IN RTL_Module
  FILTER mod.repo == 'openrisc/mor1kx.git'
    AND mod.valid_from_ts <= T
    AND mod.valid_to_ts   >= T

  // One epoch per module — pick the one whose start_ts is closest to T
  LET best_epoch = FIRST(
    FOR ep IN OUTBOUND mod BELONGS_TO_EPOCH
      SORT ABS(ep.start_ts - T) ASC
      LIMIT 1
      RETURN ep
  )
  FILTER best_epoch != null

  FOR ep2, e, p IN 1..1 OUTBOUND mod BELONGS_TO_EPOCH
    FILTER ep2._id == best_epoch._id
    RETURN p
```

Interval Query B — Change window: "what commits happened between two dates and what did they touch?"

Filters `GitCommit` on `valid_from_ts BETWEEN T1 AND T2`, keeps only commits that modified at least 2 modules (to filter out merge/tag-only commits), then returns all `MODIFIED` paths.

Demonstrates: The MOR1KX v5.1 post-release period (Jan–Apr 2022): 92 module-level changes across focused refactors — formal verification, dcache fixes, port renames, and licence updates.

Returns: paths `GitCommit → MODIFIED → RTL_Module` for every qualifying commit in the window.

```
// ────────────────────────────────────────────────────────────────────────────
// INTERVAL QUERY B — Commit activity in a date range
//   T1 / T2 define the window (inclusive).
//   FILTER cnt >= 2 skips merge/tag commits that touched zero or one module.
//
//   Useful windows for MOR1KX:
//     1327599687 → 1378100000   birth to v1.0 (all early commits)
//     1640995200 → 1651276800   Q1 2022 post-v5.1 refactoring period
//     1724578502 → 1741709888   v5.2 era (2024)
//
// Graph tab shows: GitCommit hubs each radiating to the modules they touched
// ────────────────────────────────────────────────────────────────────────────
LET T1 = 1640995200   // ← 2022-01-01
LET T2 = 1651276800   // ← 2022-04-30

FOR c IN GitCommit
  FILTER c.repo == 'openrisc/mor1kx.git'
    AND c.valid_from_ts >= T1
    AND c.valid_from_ts <= T2

  LET cnt = LENGTH(FOR m IN OUTBOUND c MODIFIED RETURN 1)
  FILTER cnt >= 2   // skip merge/no-op commits

  FOR v, e, p IN 1..1 OUTBOUND c MODIFIED
    RETURN p
```

Interval Query C — New arrivals: "which modules were introduced during a date range?"

Uses the `BELONGS_TO_EPOCH` edge field `role == 'introduced_in'` combined with a time-window filter to find modules that first appeared within `[T1, T2]`. `COLLECT` deduplicates by module label so each logical module appears once even if it crossed multiple epoch boundaries.

Demonstrates: Between the first commit and v1.0 (2012 → 2013), 29 new modules were introduced — including all three pipeline variants: cappuccino, espresso, and prontoespresso.

Returns: one path per newly introduced module — `RTL_Module → BELONGS_TO_EPOCH → DesignEpoch`.

```
// ────────────────────────────────────────────────────────────────────────────
// INTERVAL QUERY C — Modules introduced in a time window
//   Finds modules whose BELONGS_TO_EPOCH.role == 'introduced_in' edge falls
//   inside [T1, T2]. COLLECT deduplicates by label (one path per module).
//
//   Useful windows:
//     1327686400 → 1378100000   post-birth growth to v1.0 (29 new modules)
//     1378100000 → 1493300000   v1.0 → v5.0 era (any new additions)
//     1640995200 → 1653253288   v5.1 cycle
//
// Graph tab shows: newly introduced RTL_Module nodes fanning out to their
//                  introduction epochs — layout reveals which refactor introduced each module
// ────────────────────────────────────────────────────────────────────────────
LET T1 = 1327686400   // ← 2012-01-27 (day after first commit)
LET T2 = 1378100000   // ← 2013-09-02 (just after v1.0)

FOR e IN BELONGS_TO_EPOCH
  FILTER e.role        == 'introduced_in'
    AND e.valid_from_ts >= T1
    AND e.valid_from_ts <= T2

  LET mod = DOCUMENT(e._from)
  FILTER mod.repo == 'openrisc/mor1kx.git'

  COLLECT lbl = mod.label INTO grp KEEP e, mod

  LET first_e   = grp[0].e
  LET first_mod = grp[0].mod
  LET epoch     = DOCUMENT(first_e._to)

  RETURN {
    vertices: [first_mod, epoch],
    edges:    [first_e]
  }
```

Tips for the ArangoDB Graph tab

- **Hover** on any `RTL_Module` node to see `file_hash`, `valid_from_ts`, `valid_to_ts`, and `design_epoch` — these tell the temporal story of that snapshot.
- **Hover** on any `BELONGS_TO_EPOCH` edge to see `role` (`"introduced_in"` vs `"modified_in"`).
- **Hover** on any `CROSS_REPO_EVOLVED_FROM` edge to see `lineage` and `confidence`.
- For Query 2, 76 spokes can be dense — use **Settings** → **Force** to spread them out.
- Unix timestamps: multiply by 1000 and parse as milliseconds, or use `DATE_FORMAT` in AQL for human-readable dates: `DATE_FORMAT(mod.valid_from_ts * 1000, '%yyyy-%mm-%dd')`.